

Ejercicio 5: Luces y texturas

Partiendo del código generado en la práctica 4, se pide crear un programa que muestre una ventana de OpenGL con dos objetos rotando sobre sí mismos, coloreados con texturas e iluminados.

Código fuente de shaders de vértices y fragmentos

Modificar los programas de GPU para que puedan calcular luces de tipo puntual a nivel de fragmento, y poder colorear el objeto usando texturas.

Clase Program

Se debe modificar la clase “Program” para que contenga métodos de inicialización de las variables necesarias para poder simular luz y texturas de color (similar a lo visto en clase):

- Datos de normales
- Tipo/posición/color/dirección de una luz y constantes de iluminación
- Datos de brillo y textura de un material

Clase Light:

Se debe crear una clase “Light” que sirva para almacenar las propiedades de un objeto de tipo “luz”. Esta clase deberá tener los siguientes métodos/atributos:

- Vector4f direction, position, color;
- float ka, kd, ks;
 - o Constantes de iluminación ambiental, difusa y especular, respectivamente
- Light(Vector4f direction, Vector4f position, Vector4f color, float ka, float kd, float ks)
 - o Constructor de objeto de tipo luz, que recibe por parámetros los valores de sus atributos
- void move(double timeStep):
 - o Método que actualiza la posición de la luz en función de un algoritmo de movimiento (por el momento, vacío)

Clase Texture:

Se debe crear una clase que almacene las propiedades de la textura con el que se dibujará el objeto. Esta clase debe contener los siguientes atributos/métodos:

- std::string textureName;
 - o Nombre del fichero de textura que se usará para colorear el objeto
- unsigned int textureId;
 - o Identificador de la textura cargada en OpenGL

- int width,height;
 - o Tamaños en píxeles de la textura cargada
- vector <unsigned char> rawImage;
 - o Datos de color de la imagen que se usará como textura
- Texture(string fileName);
 - o Constructor de la clase usando ficheros de imagen. Carga la imagen en el vector rawImage usando la librería stbi e inicia el identificador de opengl
- void updateTexture();
 - o Copia los datos de textura a GPU

Clase Material

Esta clase controla los datos del material del objeto, en concreto propiedades de iluminación y textura de color usada. Tendrá los siguientes atributos/métodos:

- bool usaTextura
 - o Atributo para compatibilidad con objetos sin textura. El shader lo comprobará para saber qué color usar (color de vértice o textura)
- int shiny
 - o Capacidad de pulido/brillo del objeto. Mínimo “1”
- float alpha
 - o Cantidad de transparencia/opacidad propia del objeto, entre 0 y 1
- float Ka,Kd,Ks
 - o Constantes de iluminación del material
- Texture* texture;
 - o Instancia de clase textura para el color de los objetos. Se cargará desde la clase shader para subir los datos a GPU
- Material(std::string textureName,float ka, float kd, float ks)
 - o Constructor de la clase, a partir de un nombre de fichero de textura y sus constantes de iluminación. Una vez invocado, cargará los datos de textura del fichero que recibe por parámetros, usando los métodos de opengl vistos en clase.

Clase Object3D:

Añadir un atributo “Material* mat;” que contendrá el material de renderizado de este objeto. Modificar el método “loadFromFile” para leer las características de textura/material del objeto cargado, e inicializar ese atributo con sus datos de textura.

Clase Render:

Añadir un atributo de tipo “light” y un método “put/getLight” para poder acceder a él. Este atributo será inicializado desde el programa Main, guardando los datos de un tipo de luz.

- void drawGL()
 - o Modificarlo para que use los datos de luz de la escena y el material del objeto a la hora de dibujarlo.

- MainLoop()
 - o Modificarlo para que, antes de dibujar los objetos, llame al método "move" de las luces que se hayan añadido al Render.

Programa principal:

- Usando las clases creadas anteriormente, se pide implementar un programa que interactúe con el usuario por teclado y ratón, y que realice las siguientes operaciones:
- **Parte 1:**
- Usando las herramientas vistas en clase (Blender y plugin exportador), crear un modelo de una esfera texturada con un color plano (no hace falta una textura de planeta real), y un cubo texturado con la textura "cubeText.png" dada en clase.
- **Parte 2:**
- Crear una clase "Render" de tamaño 640*480 píxeles, y llamar a su método "initGL"
- Crear una variable "esfera" de tipo Object3D, y llamar a su método "loadFromFile" pasándole el nombre del fichero "sphere.frs" creado anteriormente. Debe tener un tamaño de una unidad de opengl de ancho/alto/fondo. Colocarla en la posición (0,0,0).
- Crear una clase "Cubo" heredera de tipo Object3D, y llamar a su método "loadFromFile" pasándole el nombre del fichero "cubeText.frs" creado anteriormente. Debe tener un tamaño de una unidad de OpenGL de lado. Colocarlo en la posición (3,0,0)
- Modificar el método "move" de la clase Cubo para que rote alrededor del punto (0,0,0) a una velocidad de 45º por segundo (aproximadamente)
- Crear una cámara, posicionarla en la coordenada <0,0,6> y que apunte a la coordenada <0,0,0>. Ángulo de apertura de 90º, planos zNear=0.01, zFar=100
- Crear una luz blanca de tipo puntual, y posicionarla en las coordenadas (0,0,0)
- Añadirlos al render (putObject,putCamera,putLight), y mostrar el buffer por ventana (mainLoop)
- Cuando se pulsen las teclas "Up" , "Down" ,"Left" "Right" (flechas del teclado), la cámara se moverá acorde con las teclas pulsadas.

Evaluación:

La práctica se considerará aprobada si al menos carga un archivo en formato FRS como el usado en clase (un cuadrado con dos triángulos), con una textura simple y que al rotar sobre sí mismo pueda verse los cambios de iluminación. Además, deberá usar los shaders vistos en clase para renderizar el escenario.

Entrega:

Se debe entregar el código desarrollado a través del blackboard antes de la fecha de entrega dada. Si se entrega más tarde, la práctica tendrá una penalización del 50% del total de la puntuación. En caso de trabajar en grupo ambos alumnos deben entregar la práctica a través del blackboard.

Nombrado del fichero:

Entregar un fichero en formato zip siguiendo la siguiente nomenclatura:

- PRG5_NOMBREAPELLIDO1_NOMBREAPELLIDO2.zip

La práctica debe defenderse ante el profesor. Si se realiza en parejas, ambos alumnos deben estar presentes. Las fechas de defensa se comunicarán una vez realizadas las entregas.